Universidad Mariano Gálvez de Guatemala
Facultad de Ingeniería en Sistemas y Ciencias de la Computación
Ing. Erwin Geovani De León Chaman
Programación 2

Proyecto Final Programación 2

Ángel Ignacio Tot Ordoñez	0902-25-12943
Jefferson Ivan Vicente Castro	0902-19-26304
Franklin Ismael Tilom Cojoc	0902-25-20141

Cobán, Alta Verapaz, noviembre del 2026.

Introducción

El presente documento expone el diseño arquitectónico, el modelado de datos y la implementación final del Sistema de Gestión Escolar (SGE). El propósito fundamental del sistema es ofrecer una solución de software integral y robusta para la administración académica y operativa de un establecimiento educativo, abarcando desde el registro de estudiantes, docentes y padres o encargados, hasta la configuración de grados y secciones, asignación de cursos, matriculación por ciclo escolar, control diario de asistencia y emisión de boletas de calificaciones con cálculo automatizado de promedios ponderados.

El proyecto se construyó de manera incremental aplicando los pilares de la Programación Orientada a Objetos, el diseño de bases de datos relacionales y el uso de patrones de diseño como Modelo-Vista-Controlador (MVC) y Repository. En una primera etapa se validó la lógica de negocio utilizando estructuras dinámicas en memoria (listas de objetos mediante ArrayList), para posteriormente evolucionar hacia una capa de persistencia formal conectada a un motor de base de datos PostgreSQL contenerizado en Docker mediante el estándar JDBC (Java Database Connectivity), garantizando la integridad referencial, la seguridad contra inyecciones SQL y el manejo controlado de excepciones en una interfaz gráfica moderna desarrollada en JavaFX.

Problema Abordado

En numerosos centros educativos de nivel primario y medio, la gestión de expedientes estudiantiles, control de matrícula, asignación de carga académica docente y generación de boletas de calificaciones suele realizarse mediante hojas de cálculo aisladas o registros físicos manuales. Esta fragmentación de la información genera cuatro problemáticas operativas críticas:

Duplicidad e inconsistencia de datos: La falta de un repositorio centralizado provoca que los datos de un mismo alumno, encargado o docente se transcriban múltiples veces en distintos listados, incrementando los errores de captura.

Desconexión entre matrícula, grados y cursos: Sin validaciones relacionales, resulta difícil verificar en tiempo real qué estudiantes pertenecen a cada grado y sección en el ciclo escolar activo, o asegurar que cada asignatura cuente con un docente titular y un salón debidamente registrados.

Vulnerabilidad y pérdida de registros históricos: Los archivos locales carecen de integridad referencial y control transaccional, exponiendo al colegio al extravío de historiales de asistencia o registros de inscripción al finalizar el año lectivo.

Lentitud y errores en el cálculo de boletas de calificaciones: La consolidación manual de notas por curso y el cálculo de promedios ponderados por bimestre o unidad consume una gran cantidad de horas administrativas y retrasa la entrega de reportes a los padres de familia.

Para resolver de raíz esta problemática, el Sistema de Gestión Escolar (SGE) unifica la operación del colegio en cinco módulos funcionales interconectados sobre una base de datos relacional única: (1) Estudiantes e Inscripciones, (2) Cursos y Grados/Secciones, (3) Docentes y Asignación de Cursos, (4) Calificaciones y Asistencia bajo estructura Maestro-Detalle, y (5) Padres o Encargados. De esta forma, cualquier registro creado o actualizado en un módulo queda inmediatamente disponible para los demás procesos institucionales.

Objetivo

Desarrollar e implementar un Sistema de Gestión Escolar (SGE) modular de escritorio en lenguaje Java con interfaz gráfica en JavaFX y persistencia relacional en PostgreSQL mediante JDBC, aplicando el paradigma de Programación Orientada a Objetos, patrones de diseño y buenas prácticas de código para automatizar y centralizar los procesos de matrícula, organización académica, asistencia y evaluación estudiantil.

Objetivos Específicos

1. Modelar la arquitectura orientada a objetos: Diseñar e implementar un modelo de dominio en Java que incorpore herencia a través de la superclase Persona, asociaciones entre entidades académicas y una relación de composición maestro-detalle entre BoletaCalificaciones y DetalleBoleta.

2. Implementar la persistencia relacional en PostgreSQL y Docker: Construir un esquema de base de datos normalizado compuesto por 10 tablas físicas con llaves primarias (PK), llaves foráneas (FK) y restricciones de integridad, desplegado en un contenedor Docker reproducible.

3. Desarrollar operaciones CRUD, búsquedas dinámicas y reportes en interfaz gráfica: Proveer en cada módulo funcional las operaciones de Altas, Bajas y Cambios (ABCs), filtrado instantáneo por múltiples criterios y visualización tabular (grid) utilizando controles reactivos de JavaFX.

4. Aplicar patrones de diseño y mecanismos de código seguro: Estructurar la aplicación bajo los patrones MVC y Repository, integrando validaciones de entrada en formularios, sentencias preparadas (PreparedStatement) para prevenir inyección SQL y bloques try-with-resources para el manejo robusto de excepciones.

Diagrama De Clases

El diseño orientado a objetos del sistema se organiza en el paquete gt.edu.umg.gestionescolar.model y estructura la lógica de negocio en tres bloques cohesivos:

Jerarquía de Herencia (Módulo de Personas): La clase base Persona encapsula los atributos comunes (id, nombre, apellido, telefono, email) y el método utilitario getNombreCompleto(). De ella heredan Estudiante (añadiendo carnet, fechaNacimiento e idEncargado), Docente (añadiendo codigoEmpleado y especialidad) y Encargado (añadiendo cui, parentesco y direccion), eliminando la duplicación de código.

Núcleo Académico y Matriculación: Integra las clases CicloEscolar, GradoSeccion, Curso e Inscripcion. La clase Inscripcion actúa como entidad asociativa y transaccional en memoria, vinculando objetos completos de tipo Estudiante, GradoSeccion y CicloEscolar junto con la fecha de inscripción.

Evaluación y Composición Maestro-Detalle: Contempla la clase Asistencia para el control diario por curso y estudiante, y la relación de composición entre BoletaCalificaciones (clase maestra que almacena al estudiante, ciclo escolar, período, fecha de emisión y una lista interna List<DetalleBoleta>) y DetalleBoleta (clase detalle que registra cada curso evaluado, su nota, peso ponderado y observaciones, permitiendo el cálculo automático del promedio mediante el método calcularPromedio()).

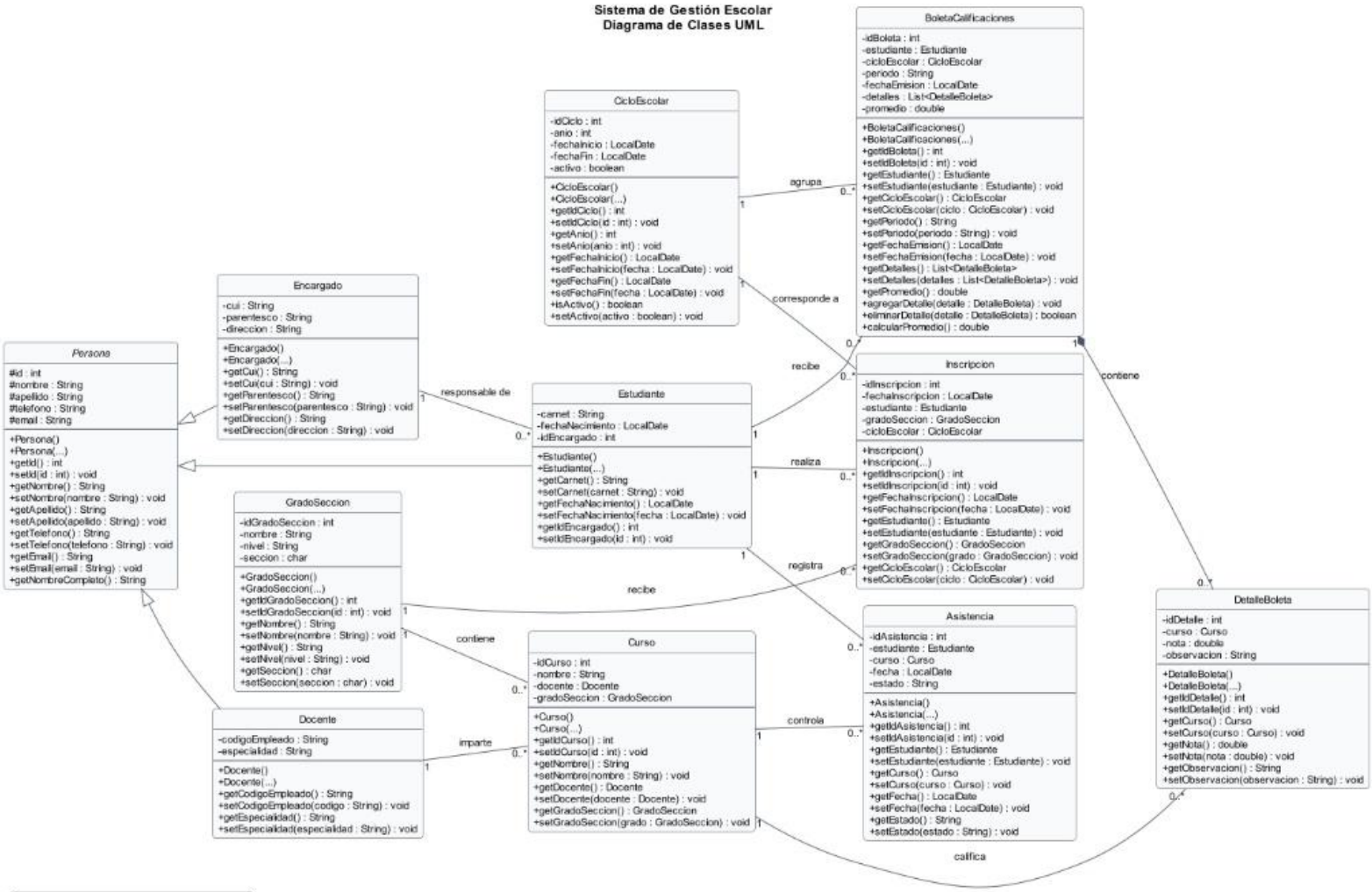
4.1 Catálogo de Entidades del Modelo de Dominio (UML)

#	Clase / Entidad	Rol en POO	Atributos Principales	Descripción y Responsabilidad
1	**Persona**	Superclase Base	id, nombre, apellido, telefono, email	Clase padre que centraliza los datos personales y promueve la reutilización mediante herencia.
2	**Estudiante**	Subclase (Hereda Persona)	carnet, fechaNacimiento, idEncargado	Representa al alumno inscrito en el plantel y mantiene la referencia hacia su padre o encargado.

3	**Docente**	Subclase (Hereda Persona)	codigoEmpleado, especialidad	Representa al catedrático en activo, habilitado para ser asignado como titular de diversos cursos.
4	**Encargado**	Subclase (Hereda Persona)	cui, parentesco, direccion	Representa al padre, madre o tutor responsable legal de uno o varios estudiantes.
5	**CicloEscolar**	Catálogo Operativo	idCiclo, anio, fechaInicio, fechaFin, activo	Define el año lectivo vigente sobre el cual se registran las inscripciones y las boletas de notas.
6	**GradoSeccion**	Catálogo Académico	idGradoSeccion, nombre, nivel, seccion	Modela el grado académico, nivel educativo (Primaria, Básico, Diversificado) y sección alfabética.
7	**Curso**	Núcleo Académico	idCurso, nombre, docente, gradoSeccion	Materia curricular asociada obligatoriamente a un objeto Docente y a un objeto GradoSeccion.
8	**Inscripcion**	Transaccional	idInscripcion, fechaInscripcion, estudiante, gradoSeccion, cicloEscolar	Formaliza la matrícula de un estudiante dentro de un grado/sección para un ciclo escolar activo.

9	**Asistencia**	Control Diario	idAsistencia, estudiante, curso, fecha, estado	Registra la presencia o ausencia de un alumno en una asignatura y fecha determinadas.
10	**BoletaCalificaciones**	Maestro (Composición)	idBoleta, estudiante, cicloEscolar, periodo, fechaEmision, promedio, detalles	Encabezado maestro de evaluación que agrupa las líneas de detalle y calcula el promedio ponderado.
11	**DetalleBoleta**	Detalle (Composición)	idDetalle, curso, nota, peso, observacion	Ítem dependiente de una boleta que almacena la calificación (0 a 100) y el peso de un curso.

4.2 Diagrama Gráfico de Clases UML



Símbolo	Significado
<	--
--	Asociación
*--	Composición maestro-detalle
1 / 0..*	Multiplicidad

Diseño De Base De Datos (Modelo Entidad-Relación)

Para garantizar la persistencia permanente y la integridad de la información, el modelo de objetos fue transformado en un esquema de base de datos relacional implementado en PostgreSQL. Al iniciar la aplicación, la clase utilitaria DatabaseManager verifica y construye automáticamente la estructura de tablas e índices mediante sentencias DDL (CREATE TABLE IF NOT EXISTS).

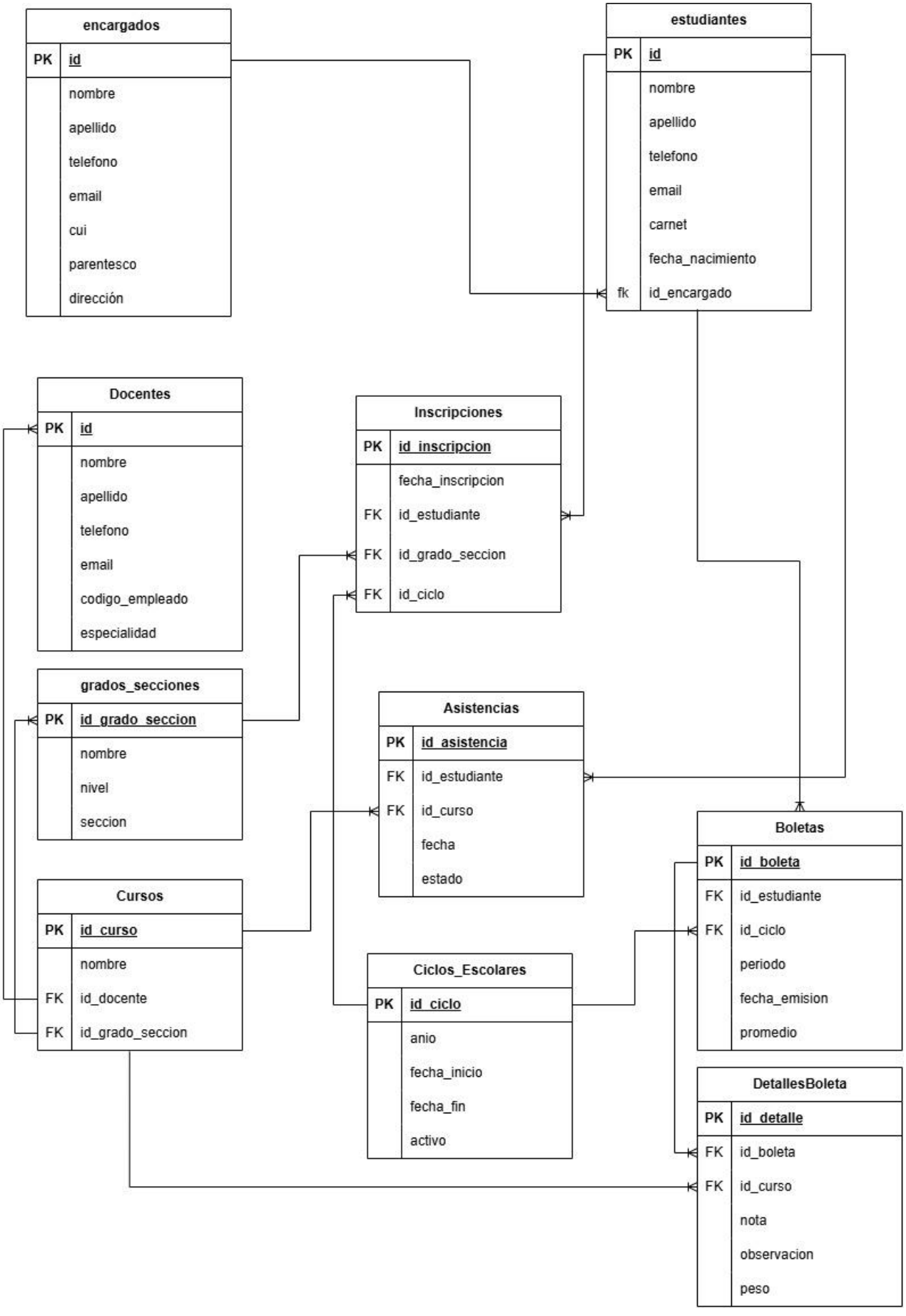
En la transición del Diagrama UML al Modelo Entidad-Relación se aplicó la estrategia de mapeo de herencia conocida como Tabla por Clase Concreta (Table per Concrete Class): dado que Persona es una superclase abstracta, sus atributos comunes (nombre, apellido, telefono, email) se materializan directamente dentro de las tablas físicas encargados, estudiantes y docentes. De este modo, el esquema relacional queda conformado por 10 tablas físicas normalizadas en Tercera Forma Normal (3FN).

Diccionario de Tablas y Relaciones del Esquema en PostgreSQL

#	Tabla Física (SQL)	Clave Primaria (PK)	Claves Foráneas (FK) y Restricciones	Propósito Relacional e Integridad
1	encargados	id (SERIAL)	cui VARCHAR(20) UNIQUE NOT NULL	Almacena a los padres o tutores legales; garantiza unicidad de CUI/DPI.
2	estudiantes	id (SERIAL)	FK: id_encargado -> encargados(id) ON DELETE SET NULL | carnet UNIQUE	Registra a los alumnos; si se elimina un encargado, el alumno conserva su expediente.
3	docentes	id (SERIAL)	codigo_empleado VARCHAR(20) UNIQUE NOT NULL	Almacena la nómina de catedráticos e impide códigos de empleado duplicados.
4	ciclos_escolares	id_ciclo (SERIAL)	anio INTEGER NOT NULL, activo BOOLEAN DEFAULT TRUE	Controla los períodos lectivos anuales (ej. Ciclo 2026) y su estado de vigencia.

5	grados_secciones	id_grado_seccion (SERIAL)	nombre, nivel, seccion CHAR(1) NOT NULL	Catálogo de aulas y grados (ej. Primero Básico - Nivel Básico - Sección A).
6	cursos	id_curso (SERIAL)	FK: id_docente -> docentes(id) FK: id_grado_seccion -> grados_secciones(id_grado_seccion)	Vincula cada materia con su docente titular y grado asignado (ON DELETE CASCADE).
7	inscripciones	id_inscripcion (SERIAL)	FK: id_estudiante -> estudiantes(id) FK: id_grado_seccion -> grados_secciones FK: id_ciclo -> ciclos_escolares(id_ciclo)	Tabla transaccional que formaliza la matrícula anual de un alumno en un grado.
8	asistencias	id_asistencia (SERIAL)	FK: id_estudiante -> estudiantes(id) FK: id_curso -> cursos(id_curso) UNIQUE(id_estudiante, id_curso, fecha)	Registra asistencia diaria e impide duplicar asistencia del mismo alumno, curso y fecha.
9	boletas_calificaciones (Boletas)	id_boleta (SERIAL)	FK: id_estudiante -> estudiantes(id) FK: id_ciclo -> ciclos_escolares(id_ciclo) UNIQUE(id_estudiante, id_ciclo, periodo)	Tabla maestra que consolida el promedio ponderado por estudiante, ciclo y período.
10	detalles_boletas (DetallesBoleta)	id_detalle (SERIAL)	FK: id_boleta -> boletas_calificaciones FK: id_curso -> cursos(id_curso) CHECK (nota BETWEEN 0 AND 100), CHECK (peso > 0)	Tabla detalle vinculada en cascada a la boleta; valida rangos de notas y pesos positivos.

Diagrama Entidad-Relación (ER)



Patrón De Diseño Aplicado

Con el objetivo de construir un software modular, escalable y fácil de mantener en equipo, se implementó el patrón arquitectónico MVC (Modelo-Vista-Controlador) complementado con el patrón Repository y una clase centralizada de gestión de conexiones (DatabaseManager).

Patrón Modelo-Vista-Controlador (MVC)

Dónde se aplicó: Se estructuró el código fuente en paquetes independientes: gt.edu.umg.gestionescolar.model (clases de dominio y reglas de negocio), gt.edu.umg.gestionescolar.view (archivos de interfaz declarativa PersonasView.fxml, AcademicoView.fxml, AsistenciaView.fxml y BoletaView.fxml estilizados con CSS) y gt.edu.umg.gestionescolar.controller (PersonasController, AcademicoController, AsistenciaController y BoletaController).

Qué problema resuelve: Desacopla por completo la presentación visual de la lógica de datos.

Patrón Repository y Gestión Centralizada de Datos

Dónde se aplicó: En el paquete gt.edu.umg.gestionescolar.repository mediante la interfaz genérica Repository<T>, la cual define las operaciones estándar save(), update(), delete(), findById(), findAll() y search(). Cada entidad cuenta con su propia implementación concreta (EstudianteRepository, DocenteRepository, EncargadoRepository, CicloEscolarRepository, GradoSeccionRepository, CursoRepository, InscripcionRepository, AsistenciaRepository y BoletaRepository) apoyada en DatabaseManager.

Qué problema resuelve: Oculta los detalles de bajo nivel de JDBC y SQL a los controladores de JavaFX. Este patrón fue clave durante el semestre porque permitió migrar el sistema desde un almacenamiento temporal en listas (ArrayList en memoria RAM durante el segundo entregable) hacia una base de datos PostgreSQL en Docker sin necesidad de reescribir la lógica de los controladores ni de las vistas.

Resultados Obtenidos

Al concluir el desarrollo y las pruebas de integración de los módulos, se obtuvo una aplicación de escritorio totalmente funcional y conectada en tiempo real a PostgreSQL. Los resultados técnicos y operativos alcanzados se resumen a continuación:

#	Módulo / Pestaña UI	Operaciones Implementadas	Resultados y Validaciones Verificadas
1	Estudiantes, Docentes y Encargados	Altas, Bajas, Cambios (ABCs), filtro instantáneo y asignación de encargado	Persistencia verificada en PostgreSQL; validación de campos obligatorios, formatos numéricos y carnés/CUI únicos.
2	Grados y Secciones	CRUD completo y buscador dinámico por nombre, nivel o sección	Alimentación automática de los selectores (ComboBox) en los módulos de Cursos y Matrícula sin reiniciar el sistema.
3	Catálogo de Cursos	CRUD relacional vinculando Docente titular y Grado/Sección	Uso de StringConverter en JavaFX para desplegar nombres legibles y preservación de llaves foráneas en base de datos.
4	Matrícula / Inscripciones	Detección automática de Ciclo Activo (2026) y matriculación de alumnos	Creación y vinculación automática del ciclo lectivo vigente; búsqueda en tiempo real por carné, alumno o grado.
5	Control de Asistencia	Registro de asistencia por alumno, curso, fecha y estado (Presente/Ausente)	Prevención de registros duplicados en la misma fecha y asignatura; filtrado ágil en tabla tipo grid.
6	Boletas de Calificaciones (Maestro-Detalle)	Gestión de encabezado de boleta, adición de cursos/notas y cálculo ponderado	Transacción maestro-detalle en JDBC (boletas y detalles_boletas) con cálculo automático del promedio ponderado.

Cumplimiento de Código Seguro y Manejo de Excepciones: Todas las interacciones con PostgreSQL se implementaron exclusivamente mediante PreparedStatement, mitigando vulnerabilidades de inyección SQL. Asimismo, se emplearon bloques try-with-resources para evitar fugas de conexiones y alertas gráficas (Alert) en JavaFX para informar al usuario ante campos vacíos o reglas de negocio incumplidas sin interrumpir la ejecución del programa.

Conclusiones

1. Integración efectiva de POO y bases de datos relacionales: La combinación de una jerarquía de clases bien estructurada en Java (herencia en Persona y composición maestro-detalle en BoletaCalificaciones) con un modelo relacional normalizado de 10 tablas en PostgreSQL permitió resolver de forma trazable y sin redundancia el flujo completo de gestión escolar.

2. Ventaja arquitectónica de los patrones MVC y Repository: Separar la interfaz gráfica (JavaFX/FXML) del acceso a datos mediante los patrones MVC y Repository facilitó la transición desde el prototipo basado en listas en memoria (ArrayList) hacia la persistencia definitiva con JDBC, permitiendo que tres desarrolladores trabajaran simultáneamente sobre el mismo repositorio de GitHub sin colisiones estructurales.

3. Robustez y portabilidad mediante Docker y código seguro: El despliegue del motor PostgreSQL mediante contenedores Docker estandarizó el entorno de ejecución entre los integrantes del equipo, mientras que el uso estricto de sentencias preparadas (PreparedStatement), restricciones SQL y manejo de excepciones aseguró una aplicación estable y resistente a entradas inválidas.

4. Impacto práctico en la gestión educativa: El Sistema de Gestión Escolar (SGE) demuestra cómo la automatización del cálculo de promedios ponderados, la validación de matrículas por ciclo activo y las búsquedas en tiempo real reducen significativamente la carga operativa administrativa y mejoran la precisión de los reportes académicos.